

Задание 7

Коллекция orders (Заказы)

Схема коллекции

```
{
  _id: ObjectId,          // Уникальный идентификатор заказа
  customer_id: ObjectId,   // Идентификатор клиента
  created_at: ISODate,     // Дата и время оформления заказа
  items: [                // Список заказанных товаров и их цену
    {
      product_id: ObjectId, // Идентификатор товара
      price: NumberDecimal  // Цена товара в заказе
    }
  ],
  status: String,         // Статус заказа
  total_amount: NumberDecimal, // Общая сумма заказа
  geo_zone: String        // Гео зона заказа
}
```

Шард ключом выбран `customer_id` т.к. основная операция быстрый поиск истории заказов пользователя. Все заказы одного пользователя будут содержаться в одном шарде.

Коллекция products (Товары)

Схема коллекции

```
{
  _id: ObjectId,          // Уникальный идентификатор товара
  name: String,           // Наименование
  category: String,       // Категория товара
  price: NumberDecimal,   // Цена
  stock: [               // Остаток товара в каждой гео зоне
    {
```

```

    geo_zone: String,          // Идентификатор геозоны
    quantity: Number           // Количество в наличии
}
],
attributes: {                 // Дополнительные атрибуты
    color: String,            // Цвет
    size: String              // Размер
}
}

```

Шард ключом выбран category категория товара т.к основная операция поиск товаров по категории так же частые обновления остатков будет локализовано в категориях. Такой шард позволит нам держать товары одной категории в одном шарде. Логика такая что скидки чаще всего делают на категории и быстро можно будет изменять цену.

Коллекция carts (Корзина)

Схема коллекции

```

{
    _id: ObjectId,             // Уникальный идентификатор корзины
    user_id: ObjectId,         // Идентификатор пользователя (null для гостей)
    session_id: String,        // session_id для гостей
    items: [                   // Список товаров
        {
            product_id: ObjectId, // Идентификатор товара
            quantity: Number       // Количество
        }
    ],
    status: String,            // "active" | "ordered" | "abandoned"
    created_at: ISODate,       // Дата и время создания
    updated_at: ISODate,       // Дата и время последнего обновления
}

```

```
    expires_at: ISODate          // Время удаления (TTL)
}
```

Шард ключом выбран `_id` Уникальный идентификатор корзины, при создании, обновлении, удалении всегда нужна корзина.

Задание 8

Метрики:

Нагрузки

Кол-во операций в секунду по шардам, если более 10 000 операций в секунду алерт.

Время выполнения запроса $>100\text{ms}$ для среднего времени, $>500\text{ms}$ для 95%

Длина очереди операция, не более 100 операций в очереди.

Использования ресурсов

Использование CPU $>70\%$ в течении 10 минут алерт.

Использование оперативной памяти $>80\%$ от доступной памяти алерт.

Распределение данных по шардам

Неравномерность распределения данных пороговое значение $>30\%$ разница между самым большим и самым маленьким.

Размер самого большого чанка не более 128 мб.

Механизмы автоматического перераспределения данных:

При перегрузке шарда электроники стоит создать автоматичеки шарды с разделением на под категории (смартфоны, ноутбуку, телевизоры) это позволит на данном этапе разгрузить полностью шард при дальнейшем масштабировании можно разделить на бренды в подкатегориях электронника → смартфоны → apple

Также можно применить `zoned tag sharding`.

Группируем физические шарды в логические зоны. Например, выделяем два шарда для «горячих» данных.

bash

```
sh.addShardTag("shard1", "HIGH_LOAD")
```

```
sh.addShardTag("shard2", "HIGH_LOAD")
```

```
sh.addShardTag("shard3", "LOW_LOAD")
```

Создадим правила привязки диапазон шард-ключа (например, category_id + subcategory_id) к тегу.

Предположим, шард-ключ {categoryId: 1, subcategoryId: 1}

Привязываем "Смартфоны" (categoryId: 1, subcategoryId: 10) к зоне HIGH_LOAD

```
sh.updateZoneKeyRange("productsDB.products",  
    { categoryId: 1, subcategoryId: 10 },  
    { categoryId: 1, subcategoryId: 11 },  
    "HIGH_LOAD")
```

Привязываем "Ноутбуки" (categoryId: 1, subcategoryId: 20) к зоне HIGH_LOAD

```
sh.updateZoneKeyRange("productsDB.products",  
    { categoryId: 1, subcategoryId: 20 },  
    { categoryId: 1, subcategoryId: 21 },  
    "HIGH_LOAD")
```

Остальные данные (например, categoryId: 2 - "Одежда") попадут в LOW_LOAD или будут балансироваться по всем шардам.

Выявление наиболее часто запрашиваемых данных и перевод их в кэш

Добавлять реплики шарда для чтения

Задание 9

Тип операции	Максимальная задержка	Обоснование
Критические операции	0 секунд (только primary)	Риск двойной продажи, финансовые операции
Операции средней важности	≤ 5 секунд	Пользовательский опыт, актуальность информации

Тип операции	Максимальная задержка	Обоснование
Некритические операции	≤ 60 секунд	Аналитика, история, архивные данные
Фоновые задачи	Не ограничено	Обработка больших данных, ETL-процессы

Коллекция **orders** (Заказы)

Операция чтения атрибута	Источник	Допустимая задержка	Обоснование
Получение текущего статуса заказа, status	Primary	0 секунд	Высокий бизнес-риск: Пользователь должен видеть актуальный статус (обработка, доставка, отмена). Неверный статус приводит к жалобам и звонкам в поддержку.
История заказов пользователя, created_at	Secondary	≤ 30 секунд	Низкий риск: История уже завершённых заказов редко меняется. Пользователь может видеть небольшую задержку без последствий.
Поиск заказов по номеру (для поддержки), _id	Primary	0 секунд	Критично для поддержки: Оператор должен видеть актуальную информацию для решения проблем клиента.
Аналитика: популярные товары, product_id	Secondary	≤ 60 секунд	Для отчётов: Не требует реального времени, используется для бизнес-аналитики и планирования закупок.

Операция чтения атрибута	Источник	Допустимая задержка	Обоснование
Экспорт заказов для бухгалтерии	Secondary	≤ 300 секунд	Фоновая задача: Выполняется раз в день/неделю, задержка не влияет на операции.

Коллекция products (Товары)

Операция чтения	Источник	Допустимая задержка	Обоснование
Проверка наличия товара при добавлении в корзину, stock	Primary	0 секунд	Высокий бизнес-риск: Риск продажи недоступного товара (overselling). Может привести к отмене заказа, недовольству клиентов.
Поиск товаров по каталогу, _id	Secondary	≤ 5 секунд	Средний риск: Пользователь может видеть устаревшую цену или описание, но это менее критично, чем наличие. Актуальность важна для рекламных акций.
Отображение детальной страницы товара	Secondary	≤ 10 секунд	Средний риск: Цены и описания могут обновляться нечасто. Исключение: товары с динамическим ценообразованием (только primary).
Получение остатков для внутренних отчётов	Secondary	≤ 60 секунд	Для аналитики: Используется для планирования закупок, не требует реального времени.
Полнотекстовый поиск товаров	Secondary	≤ 10 секунд	Поисковая функциональность: Не

Операция чтения	Источник	Допустимая задержка	Обоснование
			критично, если найдётся устаревший товар (скорее всего он будет помечен как недоступный при детальном просмотре).

Коллекция carts (Корзины)

Операция чтения	Источник	Допустимая задержка	Обоснование
Получение текущей активной корзины пользователя	Primary	0 секунд	Высокая консистентность: Корзина меняется при каждом действии пользователя. Использование устаревшей версии может привести к потере добавленных товаров.
Восстановление брошенной корзины	Secondary	≤ 30 секунд	Низкий приоритет: Корзина уже неактивна, небольшая задержка при восстановлении приемлема.
Аналитика: средний чек в корзинах	Secondary	≤ 300 секунд	Бизнес-аналитика: Используется для маркетингового анализа, не требует актуальности в реальном времени.
Поиск корзин для очистки (TTL)	Secondary	≤ 60 секунд	Фоновая задача: Автоматическая очистка старых корзин может использовать данные с задержкой.

Задание 10.1

Критерии выбора данных для Cassandra

Тип данных	Перемещать в Cassandra?	Обоснование
Заказы (orders)	Нет	Требуют атомарного обновления остатков товара. Cassandra не подойдет т.к не поддерживает ACID между разными партициями. Создание заказа должно быть согласованно с проверкой остатков.
Корзины (carts)	Да	Временные данные, высокая частота обновлений
Пользовательские сессии	Да	Геораспределённость, высокая доступность, простое чтение/запись
Товары (products)	Нет	Обновление остатков товара должно быть строго согласовано с заказами.
История заказов	Да	Данные только для добавления (append-only), временные ряды, аналитические запросы
Инвентаризация (остатки)	Нет	Лучше оставить в mongo не разделять на разные типы БД
Пользовательские профили	Нет	Нет смысла, удобнее хранить все в одной коллекции

Задание 10.2

Сущность: КОРЗИНЫ (carts)

Основная таблица:

Partition Key: (session_id, bucket_id)

bucket_id: число 0-15 (16 бакетов)

Clustering Key: cart_id (uuid)

Обоснование:

Bucketizing защищает от горячих сессий: Сессия с 100+ товарами распределяется по 16 партициям

TTL на уровне таблицы: 7 дней, автоматическое удаление старых корзин

Статические поля: `items_count`, `items_total` - агрегированные данные обновляются атомарно

Map для items: `product_id` → (`quantity`, `price`) эффективное хранение

```
CREATE TABLE carts (  
    session_id uuid,  
    bucket_id tinyint,      -- 0-15  
    cart_id uuid,  
    items map<uuid, int>,  
    expires_at timestamp,  
    PRIMARY KEY ((session_id, bucket_id), cart_id)  
) WITH default_time_to_live = 604800;
```

Сущность: СОБЫТИЯ ПОЛЬЗОВАТЕЛЕЙ (user_events)

Основная таблица:

Partition Key: (`event_date`, `event_minute`, `bucket_id`)

`event_minute`: 0-1439 (минуты в дне)

`bucket_id`: 0-99 (100 дополнительных бакетов)

Clustering Key: `event_id` (timeuuid)

Обоснование:

Экстремальное распределение: $1440 \times 100 = 144,000$ партиций в день

Расчёт нагрузки: 50,000 событий/сек = 4.32 млрд/день → ~30,000 событий/партицию

TWCS compaction: Оптимально для time-series данных

Горячие партиции невозможны: Слишком мелкое распределение

```
CREATE TABLE user_events (  
    event_date text,
```

```
event_minute smallint,    -- 0-1439
bucket_id smallint,       -- 0-99
event_id timeuuid,
event_type text,
user_id uuid,
PRIMARY KEY ((event_date, event_minute, bucket_id), event_id)
);
```

Задание 10.3

Корзины (carts) - Баланс доступности и консистенции

Hinted Handoff: Да

Read Repair Chance: 0.05 (5%)

Anti-Entropy Repair: Редко (при проблемах)

Уровень согласованности: ONE

TTL: 7 дней

Обоснование:

Временные данные - TTL 7 дней автоматически очищает

Availability важнее - лучше показать немного устаревшую корзину, чем ошибку

Минимальный Read Repair чтобы не увеличивать latency

ONE для записи/чтения - максимальная скорость

Компромисс: Возможна небольшая рассинхронизация (несколько секунд) ради скорости.

Пользовательские сессии (sessions) - Максимальная доступность

text

Hinted Handoff: Да

Read Repair Chance: 0.0 (0%)

Anti-Entropy Repair: Никогда

Уровень согласованности: ONE

TTL: 24 часа

Обоснование:

Можно пересоздать - если сессия потеряется, пользователь залогинится снова

Availability критична - пользователь должен получить доступ к сайту в любой момент

Нулевой overhead для максимальной производительности

Никакого Anti-Entropy - данные временные (24 часа)

Компромисс: Возможна потеря сессии при сбое узла ради максимальной доступности.

История заказов (order_history) - Архивные данные

text

Hinted Handoff: Да

Read Repair Chance: 0.1 (10%)

Anti-Entropy Repair: Ежемесячно

Уровень согласованности: LOCAL_ONE

Обоснование:

Данные не изменяются - только добавляются (append-only)

Не критично если архивные данные немного рассинхронизированы

LOCAL_ONE - быстрое чтение в локальном дата-центре

Ежемесячный Anti-Entropy достаточно для архивных данных

Компромисс: Возможна задержка синхронизации между DC ради производительности.